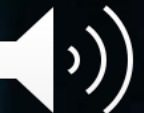




COM3021/COMM115 – Data Science at Scale

Stream Processing

DR HUGO BARBOSA



Agenda

- Past

Batch Processing

- This week

Stream processing



Feedback summary

- **Start**
 - *More applied learning:* Students want to see more practical applications of theory — particularly demonstrations of how big data and cloud systems (e.g., Docker, Kubernetes, GCP) are used in practice.
 - *Contextualized teaching:* They appreciate theoretical material but want clearer links between theory and real-world use cases, especially within the context of data science and cloud computing.
 - *Interactive learning:* Requests for more hands-on sessions, demonstrations, or workshops to complement lectures.
 - *Course materials:* Students would like slides uploaded earlier (ideally a day before) and provision of PDF notes for better preparation.
-

Feedback summary

- Stop
 - *Pacing concerns:* Some students feel the course moves too quickly through material — they'd prefer fewer rushed moments in labs.
 - *Theory-heavy emphasis:* A few comments suggest the course currently leans too much on theory without enough practical or applied components.
 - *Assessment timing:* Coursework deadline fell before all necessary material has been covered, which adds stress and reduces the chance to apply knowledge effectively.
-

Feedback Summary

- Continue
 - *Workshops and practical labs: Widely appreciated. Students value the hands-on nature and clarity of these sessions, describing them as engaging and well-prepared.*
 - *Teaching style and materials: Lectures are seen as clear, well-structured, and motivating. Students enjoy the balance of explanation and practice.*
 - *Support and enthusiasm: Several comments express gratitude for the lecturer's enthusiasm, helpfulness, and overall delivery.*
-

Introduction

- Batch processing

Read a set of files as input and produce a new set of output files.

The output is a form of derived data;

- dataset that can be recreated by running the batch process again if necessary

Assumes the data is bounded

the batch process knows when it has finished reading its input

Introduction

- Stream processing

A lot of data sources are unbounded because it arrives gradually over time:

E.g., Social media users produced data yesterday and today, and they will continue to produce more data

Batch processors could process data of fixed durations (e.g., every 24h)

We would always be looking at the past

We could make this window smaller and smaller...

... until we forget time windows and analyse every event as they arrive

i.e., Stream Processing

Event streams

- In the stream world, data records are called events.
a small, self-contained, immutable object containing details of something that happened at some point in time.
 - Usually contains a timestamp
 - Can be encoded in many ways, as a text string, JSON, binary form ...
 - Generated by a producer
AKA publisher, or sender
 - Processed by multiple consumers
AKA Subscribers or recipients
-

Event streams

- Files or databases should suffice to intermediate the interaction between consumers and producers

Querying the database or opening the file every x seconds would add overhead

- It would be better for consumers to be notified when new events appear.

Relational database management tools are not designed for that

Messaging Systems

- A common approach for notifying consumers is through a *messaging system*:
a producer sends a message containing the event, which is then pushed to consumers.
 - Multiple producer nodes can send messages
 - Multiple consumer nodes can receive them
 - *There are many approaches to this publish/subscribe model*
-

Messaging Systems

- *What happens if the producers send messages faster than the consumers can process them?*
 - *drop messages*
 - *buffer messages in a queue*
 - *backpressure (i.e., flow control)*
- *What happens if nodes crash or temporarily go offline?*

Are messages lost?

Written to disk?

Approaches

Direct messaging from producers to consumers

- Multicast over TCP or UDP
 - Consumer might be offline
-

Approaches

- **Message brokers (or message queue)**
essentially a kind of database optimized for message streams
 - Server application
 - *Producers and consumers connecting to it as clients.*
 - producers write messages to the broker
 - consumers receive them by reading them from the broker.
 - Can more easily tolerate clients that come and go (connect, disconnect, and crash),
-

Comparison

DATABASES

- Databases usually keep data until it is explicitly deleted
- Persistency
- Advanced search

MESSAGE BROKERS

- Automatically delete a messages when it has been successfully delivered
 - Queues are short
 - Some way of subscribing to a subset of topics (e.g., hashtags)
-

Multiple consumers

- When multiple consumers read messages in the same topic

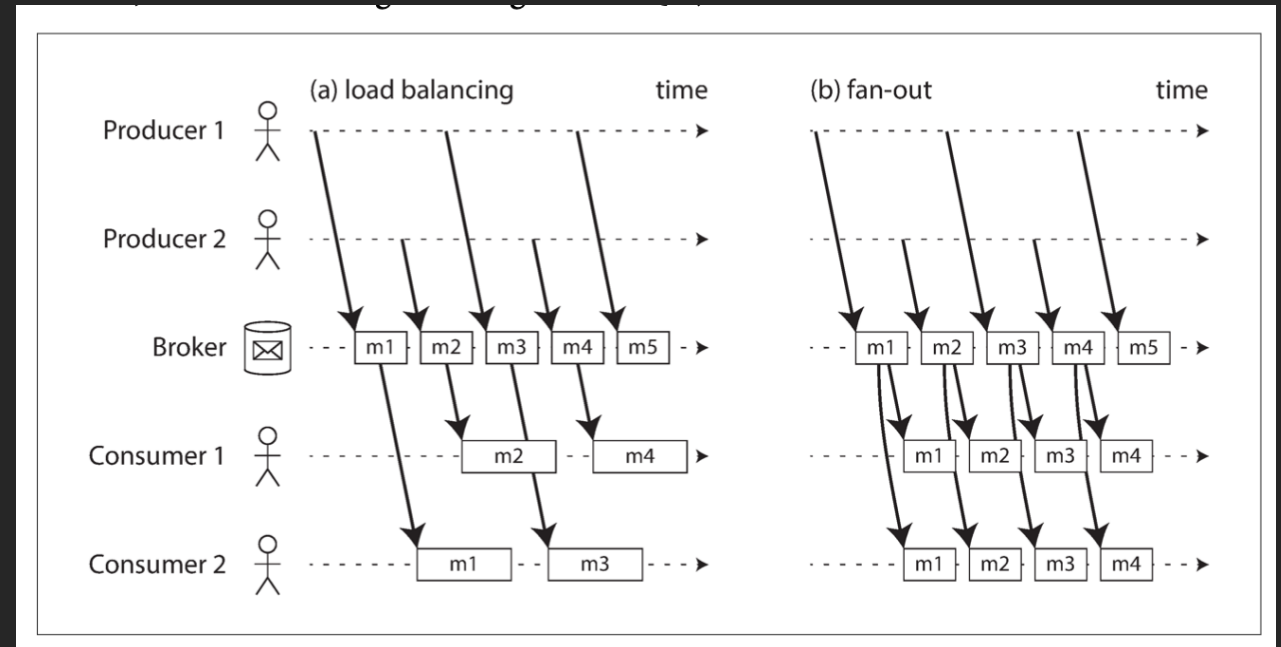
Load balancing

- Each message is delivered to one of the consumers

Consumers can share the work of processing the messages

Fan-out

- Each message is delivered to all of the consumers
allows several independent consumers to subscribe to the same broadcast of messages



Multiple consumers

- **Acknowledgments and redelivery**
 - *A client must explicitly tell the broker when it has finished processing a message*
 - *If does not receive the acknowledgement, it will deliver the message to another client*

- Can affect the ordering of messages if implemented with load balancing

